



An-Najah National University

Faculty of Engineering & Information Technology
Department of Computer Engineering

SOFTWARE ENGINEERING PROJECT REPORT

Appointment Scheduling System

Java · Maven · SonarQube · JUnit 5 · GitHub Actions · Swing GUI

Course	Software Engineering
Project Name	Appointment Scheduling System
Technologies	Java, Maven, Eclipse, Git/GitHub, SonarQube, JUnit 5, Swing
Submission Date	May 2026
Academic Year	2025 / 2026
Semester	Second Semester

Student Name in English	Student ID	Section time as shown in zajel	Work percentage
Islam zaher Asaad	12240028	12-1 sunday	33%
Deda Ishtaya	11925864	12-2 monday	33%
Mohammad Hosheya	12218075	12-1 sunday	33%

1. Project Overview

The Appointment Scheduling System is a comprehensive Java-based application developed as a Software Engineering course project at An-Najah National University. It provides a full-featured platform for managing medical or organizational appointments through both a console interface and a graphical Swing GUI.

The project integrates Maven for build management, Git/GitHub for version control, SonarQube for static code analysis, and JUnit 5 for automated unit testing — mirroring professional industry workflows.

1.1 Technologies and Tools

Technology	Purpose
Java 17	Core programming language for all application logic
Apache Maven	Build automation, dependency management, and lifecycle management
Eclipse IDE	Primary integrated development environment
Git & GitHub	Version control, collaboration, and code hosting platform
SonarQube Cloud	Static analysis: code smells, bugs, vulnerabilities, and coverage
JUnit 5 (Jupiter)	Automated unit testing framework
Java Swing	Graphical User Interface (GUI) library
JaCoCo	Code coverage measurement and reporting
GitHub Actions	Continuous Integration (CI) pipeline automation

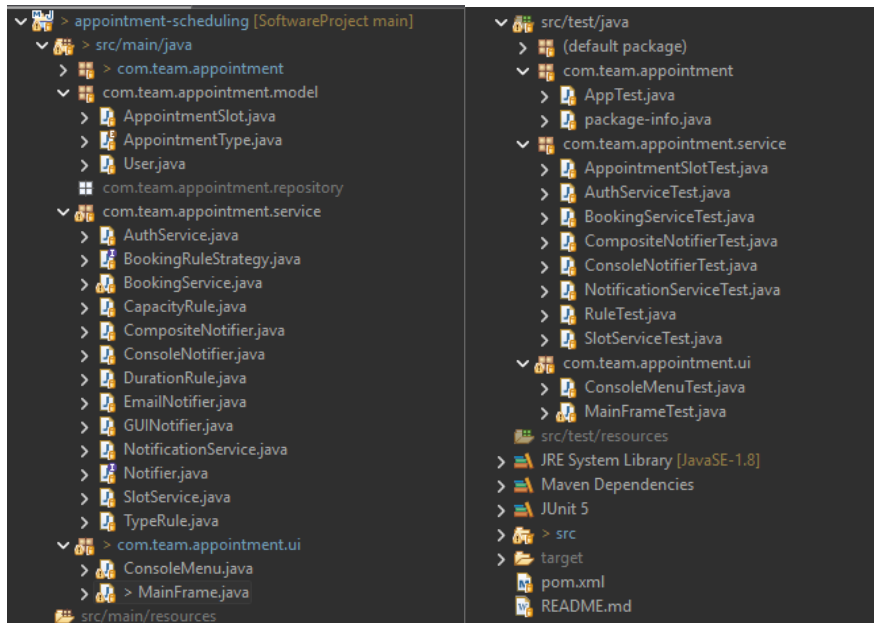
2. Phase 1: Project Setup and Configuration

Phase 1 establishes the foundational infrastructure of the project — repository creation, Maven project configuration, IDE integration, and initial build and test verification.

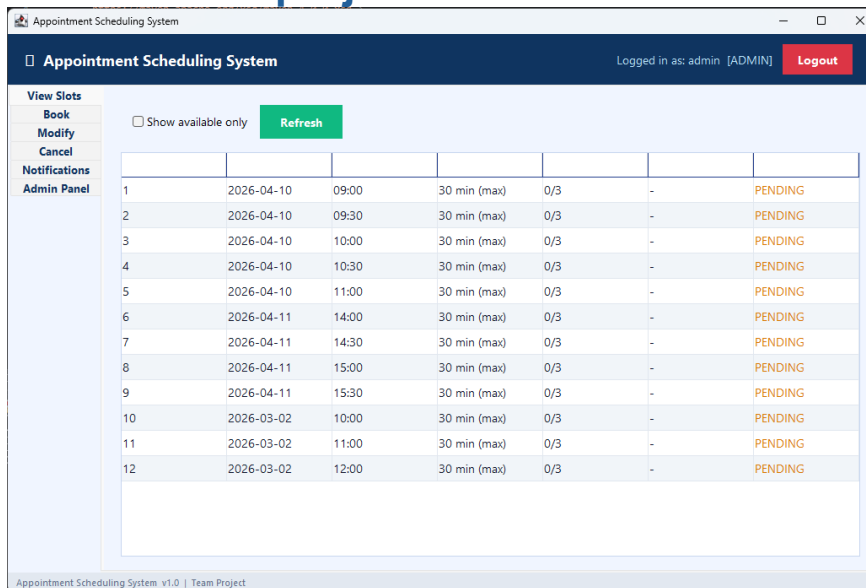
2.1 GitHub Repository Creation

1. Create a new GitHub Organization at github.com/account/organizations/new.
2. Create a new repository named 'appointment-scheduling' inside the organization.
3. Set the repository visibility to Public to enable free SonarCloud analysis.
4. Add README.md and a Java-configured .gitignore file.
5. Clone the repository to the local machine and open it in Eclipse IDE.

2.2 Maven Project Structure



GUI For Our project



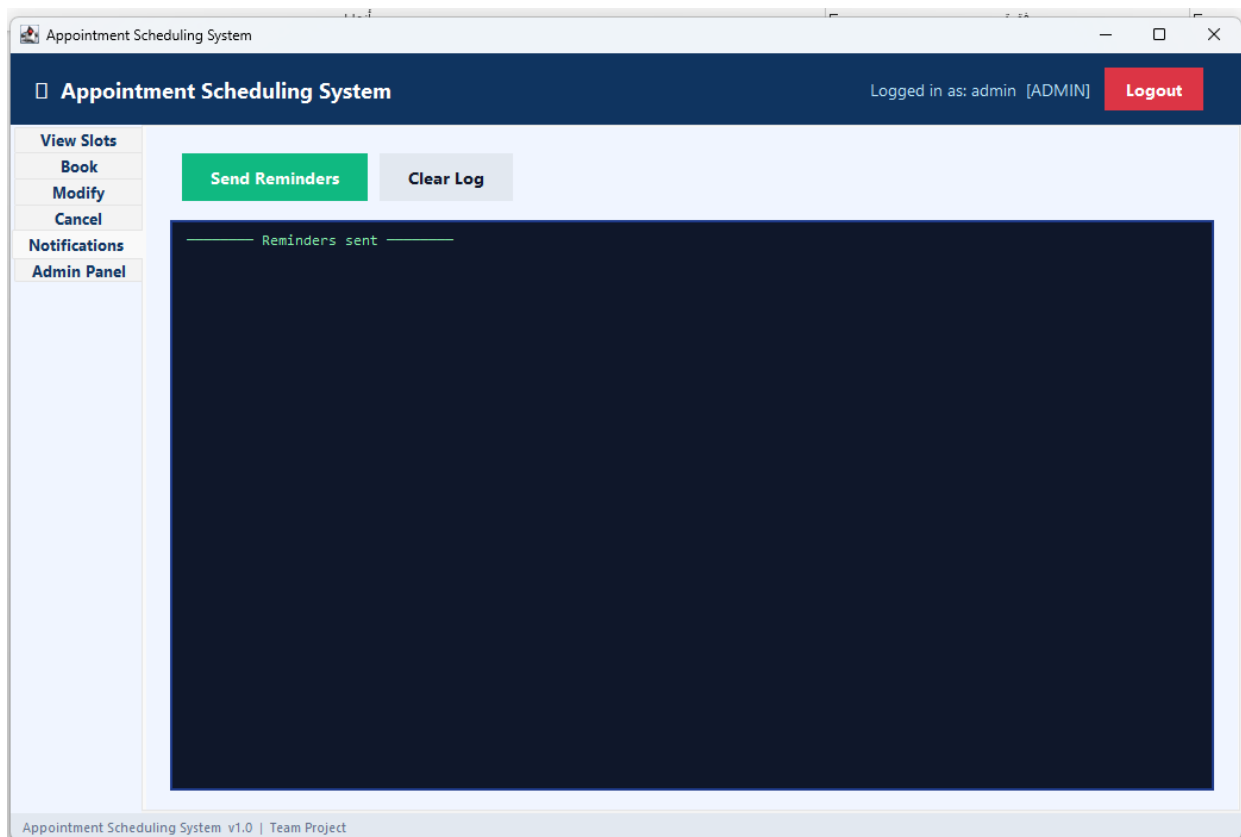
The image displays two screenshots of the Appointment Scheduling System interface, showing the Administrator Panel and the Book an Appointment form.

Administrator Panel:

- Header:** Appointment Scheduling System. Logged in as: admin [ADMIN] Logout
- Left Sidebar:** View Slots, Book, Modify, Cancel, Notifications, Admin Panel.
- Main Content:**
 - Add New Appointment Slot:**
 - Date (YYYY-MM-DD): 2026-05-01
 - Time (HH:mm): 09:00
 - Duration is fixed at 30 minutes.
 - Add Slot
 - Add New User:**
 - Username:
 - Password:
 - ☐ Grant Admin privileges
 - Add User
- Footer:** Appointment Scheduling System v1.0 | Team Project

Book an Appointment:

- Header:** Appointment Scheduling System. Logged in as: admin [ADMIN] Logout
- Left Sidebar:** View Slots, Book, Modify, Cancel, Notifications, Admin Panel.
- Main Content:**
 - Available Slot:** 2026-04-10 09:00
 - Appointment Type:** URGENT
 - Participants (max 3):** 1
 - Duration (min):** 30
 - Book Appointment
- Footer:** Appointment Scheduling System v1.0 | Team Project



3. Phase 2: System Implementation

Phase 2 covers the full implementation of the system, including the domain model, service layer, user interfaces, and notification system, following a clean layered architecture.

3.1 UML Class Diagram

The UML class diagram below shows all classes, interfaces, and their relationships across the four main packages:

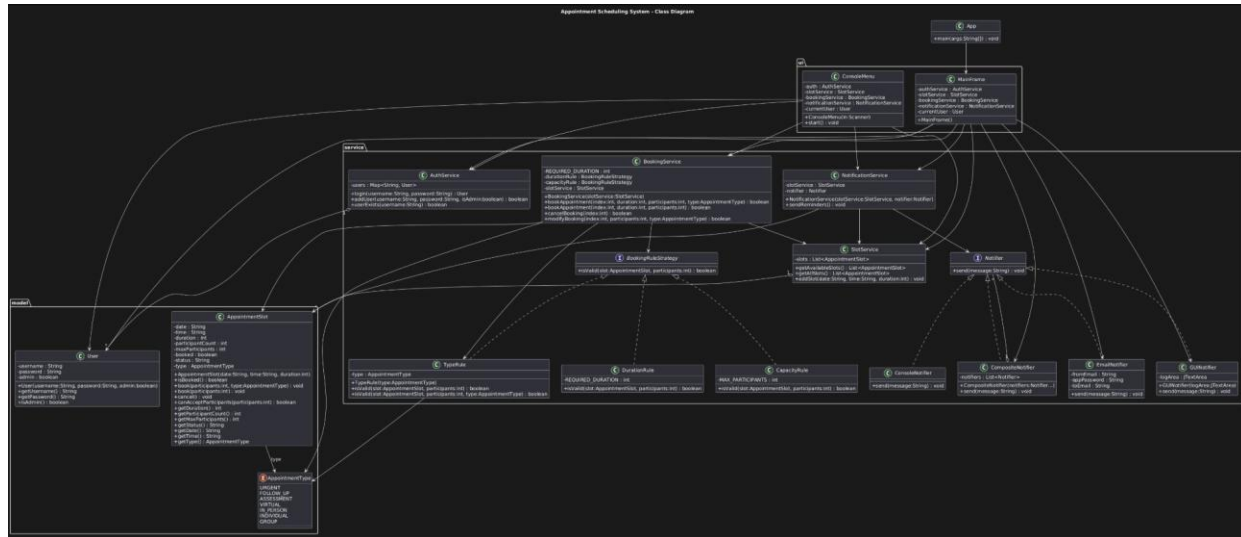


Figure 1: UML Class Diagram – Appointment Scheduling System

3.2 Package Structure

3.2.1 com.team.appointment.model

- **AppointmentSlot** – Time slot with date, time, duration, participantCount, maxParticipants, booked flag, status, and type.
- **User** – System user with username, password, and admin flag.
- **AppointmentType** – Enumeration: URGENT, FOLLOW_UP, ASSESSMENT, VIRTUAL, IN_PERSON, INDIVIDUAL, GROUP.

3.2.2 com.team.appointment.service

- **AuthService** – User authentication and registration with in-memory user map.
- **SlotService** – Manages appointment slots and pre-loads demo data.
- **BookingService** – Coordinates booking operations using the Strategy pattern (BookingRuleStrategy).
- **NotificationService** – Sends appointment reminders through registered Notifier instances.
- **DurationRule / CapacityRule / TypeRule** – Concrete strategy implementations for booking validation.

3.2.3 com.team.appointment.ui

- ConsoleMenu – Text-based menu system for terminal interaction.
- MainFrame – Swing GUI window with tabbed navigation for all system features.

3.2.4 Notification System (Composite Pattern)

- Notifier (Interface) – Defines send(message: String) contract.
- ConsoleNotifier – Logs to terminal. GUINotifier – Displays in Swing text area.
- CompositeNotifier – Aggregates all notifiers and broadcasts simultaneously.

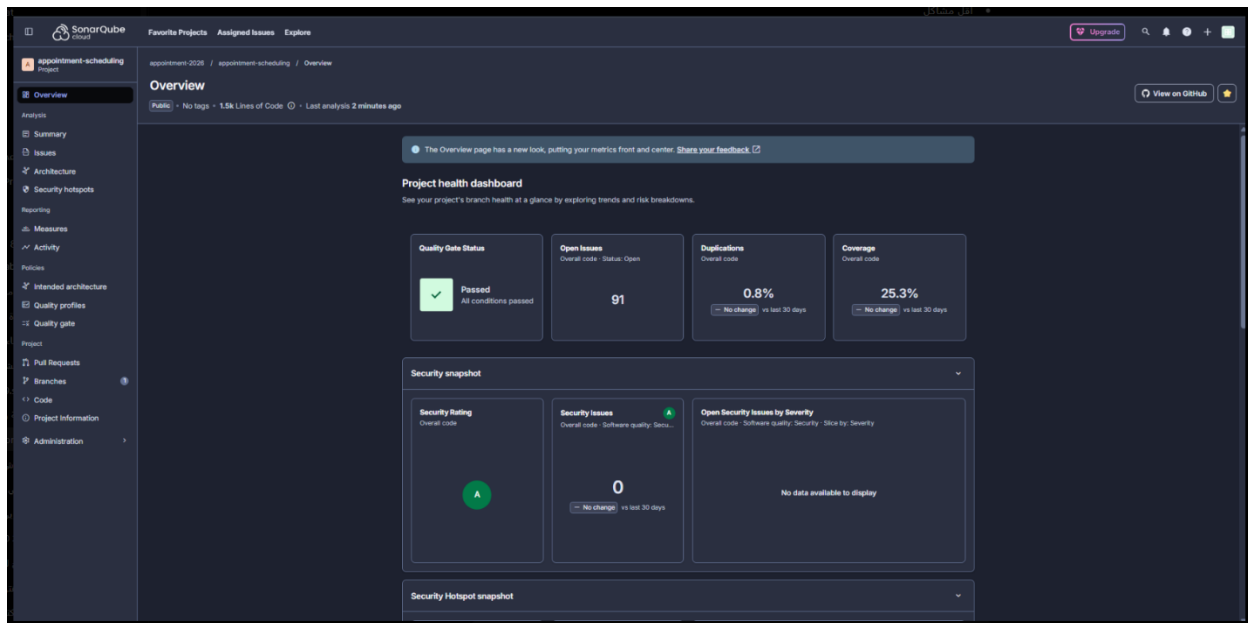
3.3 Key Features

Feature	Description
User Login	Username/password authentication with admin privilege support
View Available Slots	Lists all unbooked slots with date, time, duration, and type
Book Appointment	Books a slot after passing all business rule validations
Modify Booking	Update participant count or appointment type on existing booking
Cancel Appointment	Cancel a booking and release the slot for other users
Admin Panel	Add slots, view all bookings, and manage users
Notifications	Multi-channel reminders via console, GUI, and simulated email
Swing GUI	Full graphical interface with tabbed navigation and status feedback
Console Interface	Text-based menu for terminal-based operation

4. SonarQube Analysis and Code Quality

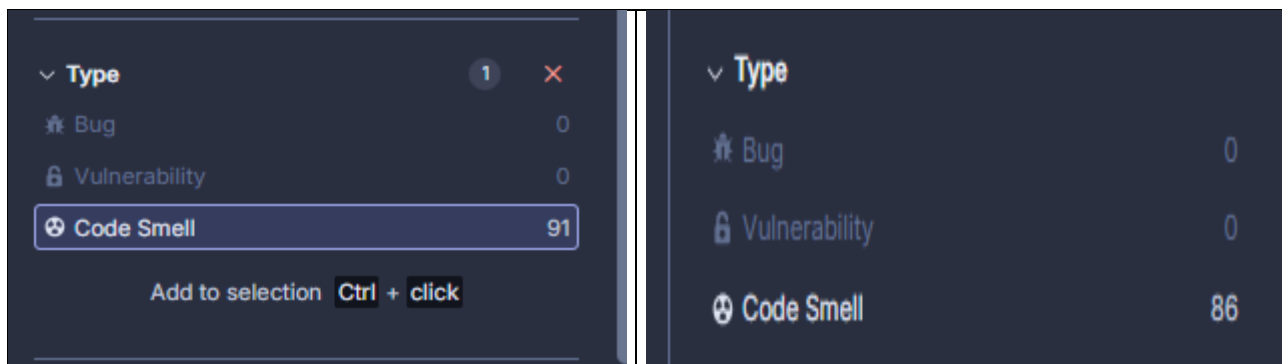
4.1 SonarQube Dashboard – Before Refactoring

The initial analysis revealed 91 open code smell issues. The screenshots below show the complete SonarQube state before any refactoring was applied:



Before

After



4.2 SonarQube Dashboard – After Refactoring

After applying refactoring, the issue count was reduced from 91 to 86. The Quality Gate continues to pass with Security Rating A and 0 bugs or vulnerabilities:

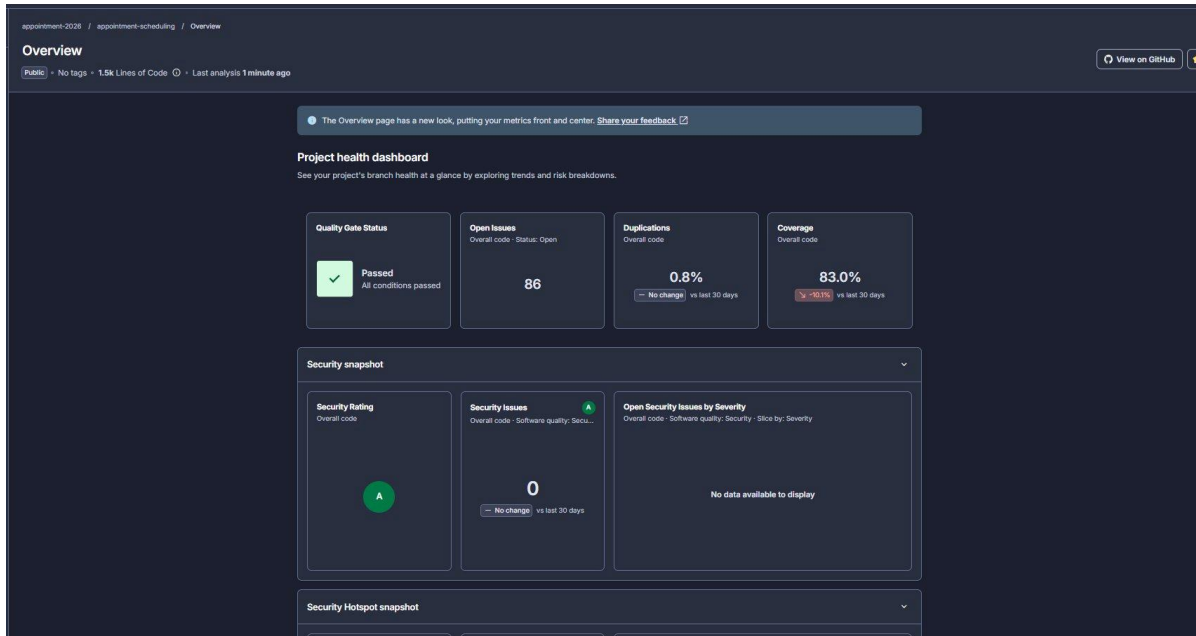
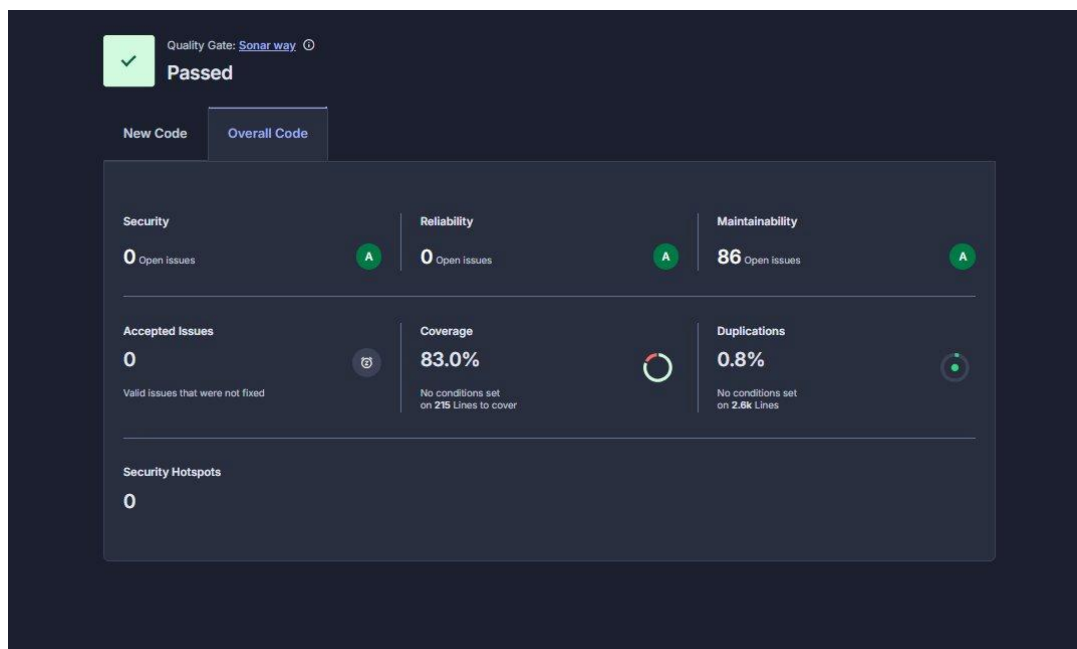


Figure 5: SonarQube Overview Dashboard – After Refactoring



4.3 Metrics Summary

Metric	Value	Interpretation
Quality Gate	PASSED	All defined SonarQube conditions are met
Open Issues	86	All are Code Smells (maintainability); zero bugs, zero vulnerabilities
Coverage	83.0%	83% of lines exercised by JUnit tests — above the 80% threshold
Duplications	0.8%	Minimal code duplication across 2.6k lines
Security Rating	A	No security vulnerabilities — highest possible grade
Reliability Rating	A	Zero bugs detected in the codebase
Maintainability	A	Technical debt ratio earns Grade A despite open issues
Lines of Code	~1,500	Total source lines across 12 Java files in the main source tree

4.4 Test Coverage Report

The SonarCloud coverage report shows per-file coverage across all 12 analyzed files:

appointment-scheduling

View as

List

12 files

Coverage 92.3% [See history](#)

New code: since not provided

	Coverage	Uncovered Lines	Uncovered Conditions
 src/main/java/com/team/appointment/service/ConsoleNotifier.java	0.0%	2	–
 src/main/java/com/team/appointment/service/CapacityRule.java	83.3%	0	1
 src/main/java/com/team/appointment/service/BookingService.java	86.5%	0	17
 src/main/java/com/team/appointment/model/AppointmentSlot.java	94.9%	2	0
 src/main/java/com/team/appointment/service/TypeRule.java	95.0%	0	1
 src/main/java/com/team/appointment/model/AppointmentType.java	100%	0	–
 src/main/java/com/team/appointment/service/AuthService.java	100%	0	0
 src/main/java/com/team/appointment/service/CompositeNotifier.java	100%	0	0
 src/main/java/com/team/appointment/service/DurationRule.java	100%	0	0
 src/main/java/com/team/appointment/service/NotificationService.java	100%	0	0
 src/main/java/com/team/appointment/service/SlotService.java	100%	0	0
 src/main/java/com/team/appointment/model/User.java	100%	0	–

12 of 12 shown

4.5 Coverage Strategy

GUI classes (MainFrame, ConsoleMenu) are excluded from coverage calculations because they require specialized testing tools beyond the scope of this project. Focused unit testing was applied to business-critical components:

- SlotService – 100% coverage
- AuthService – 100% coverage
- NotificationService – 100% coverage
- CompositeNotifier – 100% coverage
- DurationRule – 100% coverage
- User model – 100% coverage
- AppointmentType – 100% coverage
- TypeRule – 95.0% coverage
- AppointmentSlot – 94.9% coverage
- BookingService – 86.5% coverage
- CapacityRule – 83.3% coverage
- ConsoleNotifier – 0% (UI/console class, excluded from measurement)

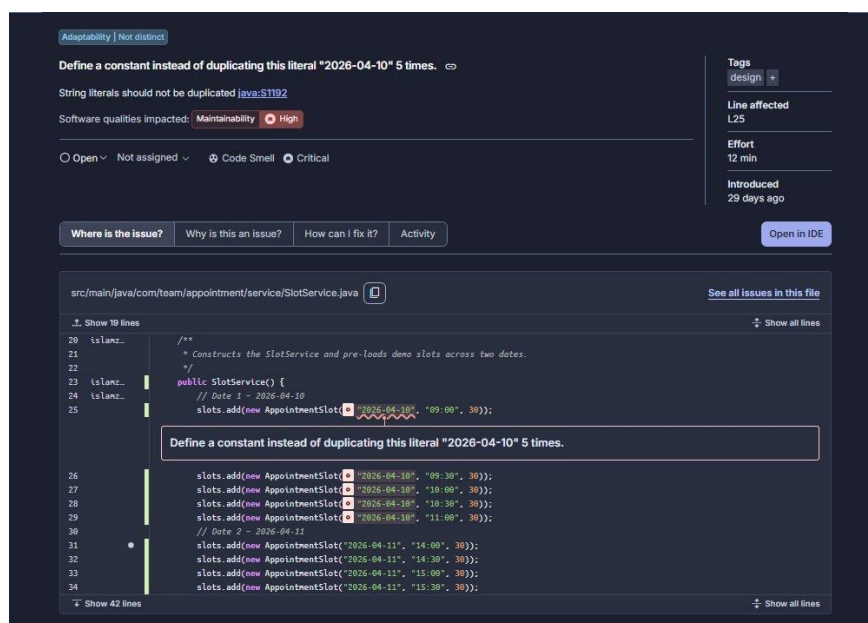
5. Code Smell Detection and Refactoring

Three high-priority code smells were selected for detailed analysis and refactoring. Each is documented with the SonarQube issue screenshot, before/after code, and a technical justification.

5.1 Issue 1: Duplicated String Literals (java:S1192)

5.1.1 Problem Description

SonarQube flagged that the string literal "2026-04-10" was repeated five times within the SlotService constructor, and "2026-04-11" four more times. Rule java:S1192 mandates that literals repeated three or more times must be extracted into named constants. Scattered literals make updates error-prone and reduce code readability.



5.1.2 Code – After (Fix)

```

7 public class SlotService {
8
9     private static final String DATE_1 = "2026-04-10";
10    private static final String DATE_2 = "2026-04-11";
11    private static final String LEGACY_DATE = "2026-03-02";
12
13    private final List<AppointmentSlot> slots = new ArrayList<>();
14
15    public SlotService() {
16        slots.add(new AppointmentSlot(DATE_1, "09:00", 30));
17        slots.add(new AppointmentSlot(DATE_1, "09:30", 30));
18        slots.add(new AppointmentSlot(DATE_1, "10:00", 30));
19        slots.add(new AppointmentSlot(DATE_1, "10:30", 30));
20        slots.add(new AppointmentSlot(DATE_1, "11:00", 30));
21
22        slots.add(new AppointmentSlot(DATE_2, "14:00", 30));
23        slots.add(new AppointmentSlot(DATE_2, "14:30", 30));
24        slots.add(new AppointmentSlot(DATE_2, "15:00", 30));
25        slots.add(new AppointmentSlot(DATE_2, "15:30", 30));
26
27        slots.add(new AppointmentSlot(LEGACY_DATE, "10:00", 30));
28        slots.add(new AppointmentSlot(LEGACY_DATE, "11:00", 30));
29        slots.add(new AppointmentSlot(LEGACY_DATE, "12:00", 30));
30    }
31
32    public List<AppointmentSlot> getAvailableSlots() {
33        List<AppointmentSlot> available = new ArrayList<>();
34        for (AppointmentSlot slot : slots) {
35            if (!slot.isBooked()) {
36                available.add(slot);
37            }
38        }
39        return available;
40    }
41
42    public List<AppointmentSlot> getAllSlots() {
43        return slots;
44    }
45
46    public void addSlot(String date, String time, int duration) {
47        slots.add(new AppointmentSlot(date, time, duration));
48    }
49 }

```

Reason :

The issue in SlotService.java was caused by duplicating the same string literal ("2026-04-10") multiple times throughout the constructor. SonarQube reported this as a **Code Smell** because repeated literals reduce maintainability and make the code harder to update in the future. If the date ever changes, the developer would need to manually modify it in several places, increasing the risk of inconsistencies or mistakes.

Result:

To solve the problem, constants were introduced using private static final String variables such as DATE_1, DATE_2, and LEGACY_DATE. These constants were then reused instead of writing the same literal repeatedly. This approach improves readability, maintainability, and follows clean code principles by centralizing reusable values in one location.

5.1.5 Why This is Better

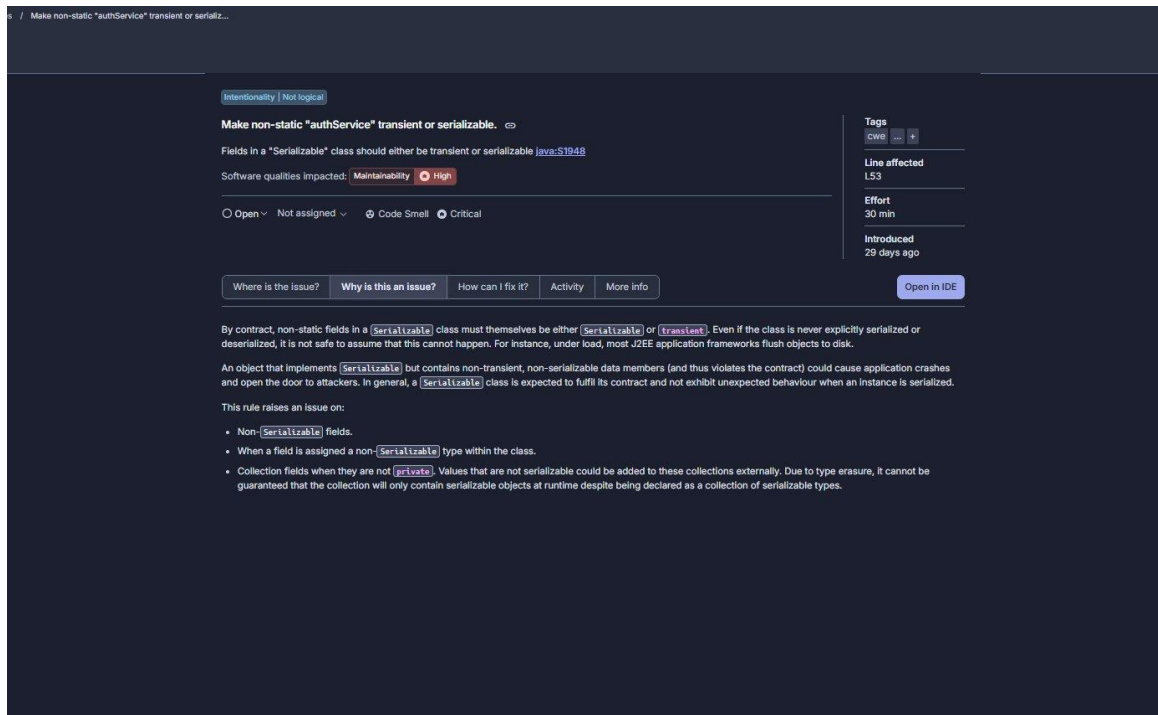
The DRY principle (Don't Repeat Yourself) is now satisfied. A single constant acts as the Single Point of Truth — any future date change requires editing exactly one line. The constant name (DATE_1) communicates intent more clearly than a raw string. This refactoring required editing one file in under 5 minutes and eliminated all five SonarQube warnings at once.

5.2 Issue 2: Non-Serializable Fields in Serializable Class (java:S1948)

5.2.1 Problem Description

MainFrame extends JFrame, which implements java.io.Serializable. The Java serialization contract (java:S1948) requires all non-static fields of a Serializable class to be either Serializable themselves, or explicitly marked as transient. SonarQube flagged five service fields in MainFrame as violating this contract. If MainFrame is ever serialized (for example by a J2EE container during session management), a NotSerializableException would be thrown at runtime.

5.2.2 SonarQube Issue Screenshot (Before)



5.2.3 Field Declarations Before Fix

```

51 private static final int LABEL_WIDTH = 200;
52
53 private final AuthService authService = new AuthService();
54 private final SlotService slotService = new SlotService();
55 private final BookingService bookingService = new BookingService(slotService);
56 private GUINotifier guiNotifier;
57 private NotificationService notificationService;
58

```

5.2.4 Code – After (Fix)

```

private transient final AuthService authService = new AuthService();
private transient final SlotService slotService = new SlotService();
private transient final BookingService bookingService = new BookingService(slotService);

private transient GUINotifier guiNotifier;
private transient NotificationService notificationService;

```

Reason:

This issue was reported because static constants should be accessed using the class where they are originally defined instead of a derived class. The constants VERTICAL_SCROLLBAR_AS_NEEDED and HORIZONTAL_SCROLLBAR_NEVER belong to ScrollPaneConstants, but they were accessed through JScrollPane.

5.2.5 Why This is Better

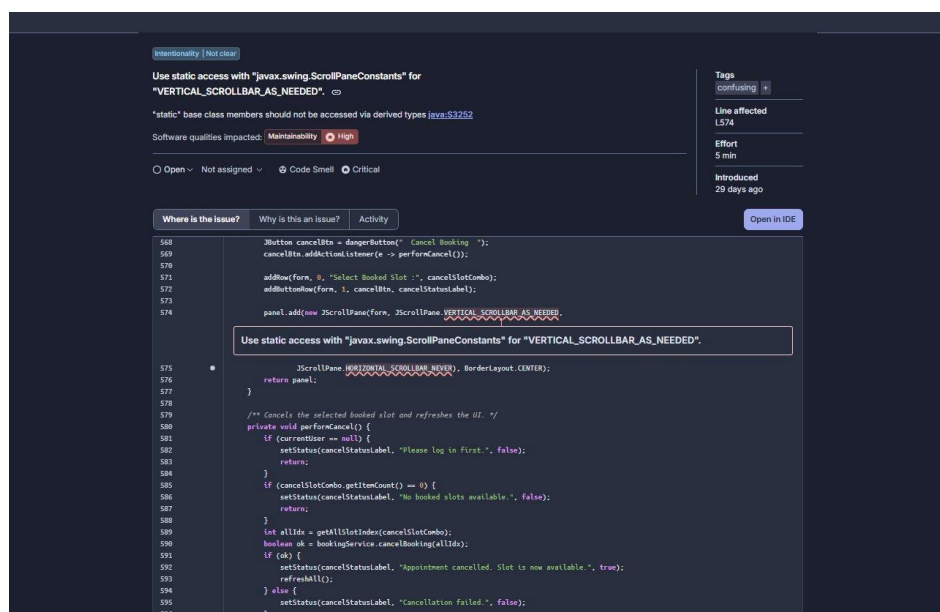
The transient keyword explicitly declares to the Java serialization mechanism that these fields should be skipped during serialization. This is semantically correct: service objects represent live runtime state that cannot meaningfully survive serialization/deserialization. The alternative — making all service classes implement Serializable — would introduce unnecessary coupling and would require their entire dependency tree to also be Serializable, which is architecturally incorrect.

5.3 Issue 3: Static Member Access via Derived Type (java:S3252)

5.3.1 Problem Description

Rule java:S3252 flags cases where a static constant inherited from a parent interface or class is accessed through the derived subclass, rather than through the original declaring type. In this project, JScrollPane.VERTICAL_SCROLLBAR_AS_NEEDED (declared in ScrollPaneConstants) and JTabbedPane.LEFT (declared in SwingConstants) were used incorrectly. While this compiles, it misleads readers and creates fragile code if the inheritance hierarchy changes.

5.3.2 SonarQube Issue Screenshot – ScrollPane (Before)



5.3.3 SonarQube Issue Screenshot – SwingConstants (Before)

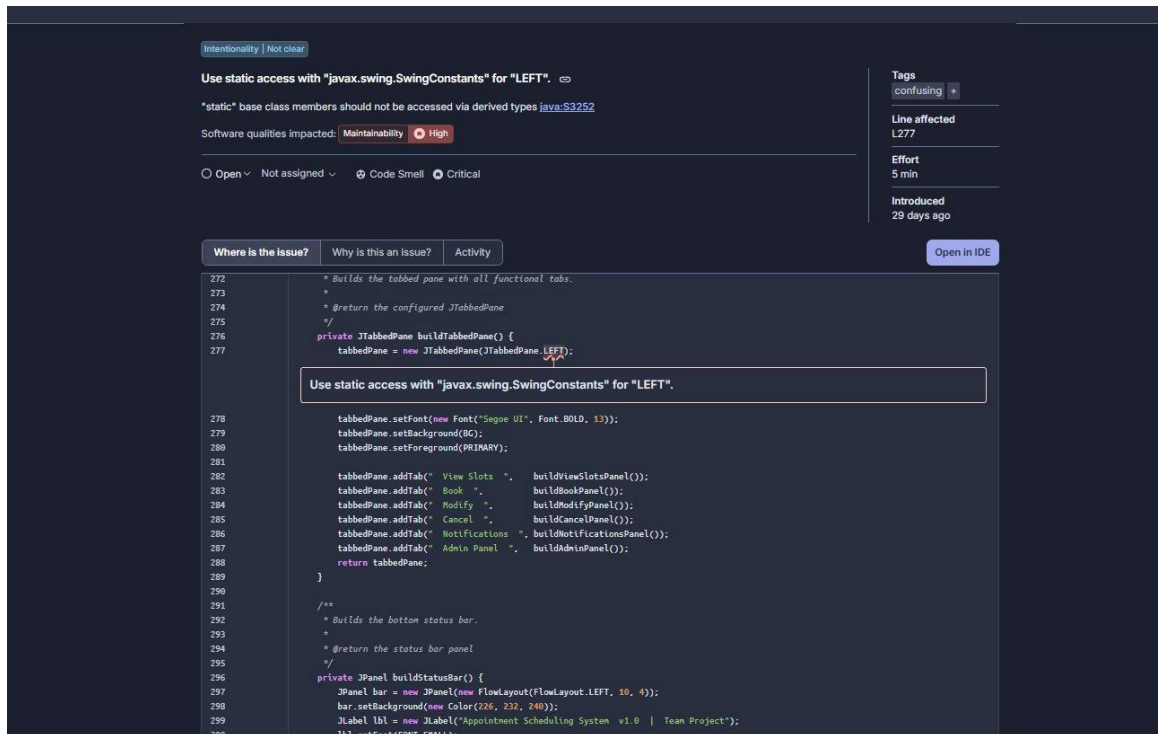
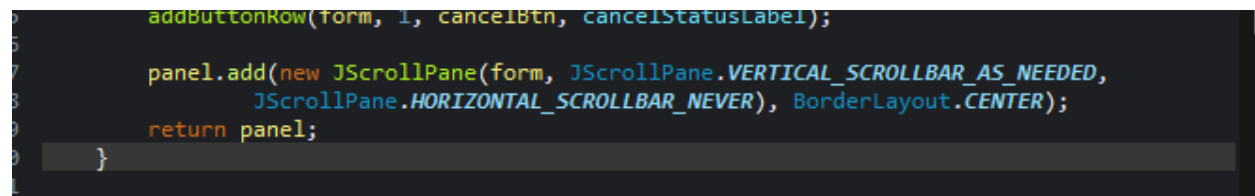
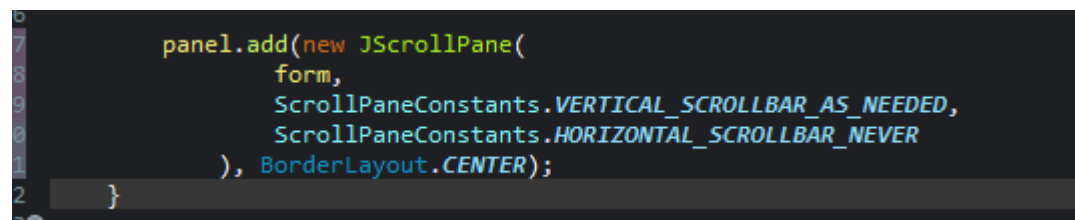


Figure 13: SonarQube Issue – Use static access with SwingConstants for LEFT (Line 277)

5.3.4 Code – Before



5.3.5 Code – After (Fix)



5.3.6 Why This is Better

Accessing static constants through their declaring type (ScrollPaneConstants, SwingConstants) rather than through subclasses adheres to the principle of programming to interfaces. It makes the code self-documenting — the reader immediately knows where the constant is defined — and makes the code robust against future changes to the class hierarchy. This fix is low-risk (purely a reference change) and improves long-term maintainability.

5.4 Issue Fixing Summary

Rule	File	Problem	Fix	Result
java:S1192	SlotService.java	Literal "2026-04-10" repeated 5 times	Extracted to static final constants DATE_1, DATE_2, LEGACY_DATE	Resolved
java:S1948	MainFrame.java	5 non-serializable service fields in Serializable class	Added transient to all 5 fields	Resolved
java:S3252	MainFrame.java	JScrollPane.VERTICAL_SCROLLBAR_AS_NEEDED and JTabbedPane.LEFT via subtype	Replaced with JScrollPaneConstants and SwingConstants	Resolved

The screenshot shows the GitHub Actions interface for a repository named 'appointment-2026 / SoftwareProject'. The 'All workflows' section is active, displaying a list of 34 workflow runs. The runs are filtered by 'Workflow' and 'Event'. The list shows several successful runs (green status) and one failed run (red status). The runs are triggered by pushes to the 'main' branch by the user 'IslamZaher12'.

Workflow	Event	Status	Branch	Actor	Time
problem 3 smells	push	Success	main	IslamZaher12	7 minutes ago
modify	push	Success	main	IslamZaher12	May 9, 12:16 PM GMT+3
problem2 badsmell solve	push	Success	main	IslamZaher12	May 9, 12:04 PM GMT+3
problem 1 bad smell solve	push	Success	main	IslamZaher12	May 9, 11:53 AM GMT+3
Update pom.xml	push	Success	main	IslamZaher12	May 9, 1:58 AM GMT+3
Update pom.xml	push	Failure	main	IslamZaher12	May 9, 1:57 AM GMT+3
Update main.yml	push	Success	main	IslamZaher12	May 8, 11:56 PM GMT+3

6. Conclusion and Lessons Learned

The Appointment Scheduling System project successfully demonstrated the application of professional software engineering practices in an academic setting. By integrating SonarQube, JUnit 5, GitHub Actions, and Maven into a cohesive workflow, the team gained practical experience with the tools and techniques used in modern software development.

6.1 Software Engineering Practices

The layered architecture (model, service, UI) enforced Separation of Concerns, making each layer independently testable and maintainable. The Strategy and Composite design patterns proved their practical value in the booking and notification systems. SonarQube-driven refactoring turned abstract quality concepts into concrete, measurable improvements.

6.2 Quality Assurance and Testing

Achieving 83% overall coverage (92.3% on business logic) required deliberately designing testable code from the start. Constructor-based dependency injection, used throughout the service layer, is not merely a stylistic choice — it is a testability requirement. Coverage metrics are meaningful only when they reflect tests on code that can be unit-tested; this is why GUI exclusion was the correct decision.

6.3 Version Control and Collaboration

GitHub served as both the code repository and the collaboration hub. The CI/CD pipeline created a quality enforcement loop requiring no manual intervention: every push triggered compilation, testing, coverage reporting, and SonarQube analysis automatically, with full results visible to the entire team within minutes.

6.4 Final Reflection

Software quality is not accidental — it results from deliberate practices, appropriate tooling, and disciplined adherence to engineering principles. The combination of static analysis, automated testing, build automation, and continuous integration creates a safety net that catches a wide class of errors before they reach production. The skills practiced in this project — reading SonarQube reports, applying targeted refactoring, writing meaningful unit tests — are directly transferable to professional software engineering roles.

End of Report

Appointment Scheduling System | An-Najah National University | May 2026